

Module 9: final keyword, upcasting and downcasting, abstract keyword, Inner class & Anonymous inner class

Q1. Final Variable

```
class Circle {  
    final double PI = 3.14;  
  
    double calculateArea(double radius) {  
        // Calculate and return area of circle  
    }  
}
```

Task: Complete the `calculateArea` method. Then try modifying the value of `PI` inside the method. What error do you get?

Q2. Final Method

```
class Animal {  
    final void makeSound() {  
        System.out.println("Some sound");  
    }  
}  
  
class Dog extends Animal {  
    // Try to override makeSound() here  
}
```

Task: Try to override the `makeSound()` method in the `Dog` class. What is the compiler's response?

Q3. Final Class

```
final class Vehicle {  
    void start() {  
        System.out.println("Vehicle starting");  
    }  
}  
  
// Try to extend Vehicle class
```

Task: Try to create a subclass of `Vehicle`. Observe and explain the error you get.

2. Upcasting and Downcasting

Q4. Upcasting Example

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

Task:

1. Create a `Dog` object.
 2. Upcast it to an `Animal` reference.
 3. Try to call `bark()` from the upcasted reference.
-

Q5. Downcasting Example

```
Animal a = new Dog(); // Upcasting
Dog d = (Dog) a;      // Downcasting
```

Task:

Write a program that:

- Performs upcasting and downcasting.
- Calls both `sound()` and `bark()` after downcasting.

Bonus: What happens if you try to downcast an object of `Animal` that's not really a `Dog`?

3. `abstract` Keyword

Q6. Abstract Class Example

```
abstract class Shape {
    abstract double area();
}

class Rectangle extends Shape {
    double length, breadth;

    Rectangle(double l, double b) {
        length = l;
        breadth = b;
    }

    // Override area()
}
```

Task: Complete the `Rectangle` class by overriding the `area()` method. Create an object and print the area.

Q7. Abstract Class with Constructor

```
abstract class Person {  
    String name;  
  
    Person(String name) {  
        this.name = name;  
    }  
  
    abstract void display();  
}
```

Task: Create a class `Student` that extends `Person` and implements `display()`. Create an object and test it.

4. Inner Class

Q8. Member Inner Class

```
class Outer {  
    private String msg = "Hello from Outer";  
  
    class Inner {  
        void display() {  
            // Print msg from Outer class  
        }  
    }  
}
```

Task: Complete the `display()` method. From `main()`, create an object of `Inner` and call `display()`.

Q9. Static Nested Class

```
class Outer {  
    static class StaticInner {  
        void show() {  
            System.out.println("Static Inner Class");  
        }  
    }  
}
```

Task: Create the `StaticInner` object from `main()` and call `show()`.

5. Anonymous Inner Class

Q10. Anonymous Class with Interface

```
interface Greeting {  
    void sayHello();  
}
```

Task:

Create an anonymous inner class that implements `Greeting` and override `sayHello()` to print `"Hello, student!"`.

Q11. Anonymous Class with Abstract Class

```
abstract class Vehicle {  
    abstract void start();  
}
```

Task:

Create an anonymous inner class that extends `Vehicle` and overrides `start()` to print `"Vehicle started anonymously"`.